UPF
UNIVERSIDADE
DE PASSO FUNDO

MINICURSO · SENAINFO 2026

MicroPython na ESP32

Do primeiro LED a um sensor de estacionamento — programando uma placa com Python.

Matheus Sebastiani Silva
Gustavo Luiz Rosset
Brenda Slongo Taca
Engenharia de Computação · Universidade de Passo Fundo

Roteiro da noite

duração total: 2h30

19:00	Microcontrolador, ESP32, MicroPython e seus módulos	15 min
19:15	Instalar e configurar o Thonny, REPL e Python	25 min
19:40	Montagem + machine.Pin: acender LEDs	20 min
20:00	machine.PWM: brilho e som	15 min
20:15	Intervalo	10 min
20:25	machine.ADC: lendo o potenciômetro	20 min
20:45	Prática final: sensor de estacionamento	35 min

O que tem na sua bancada

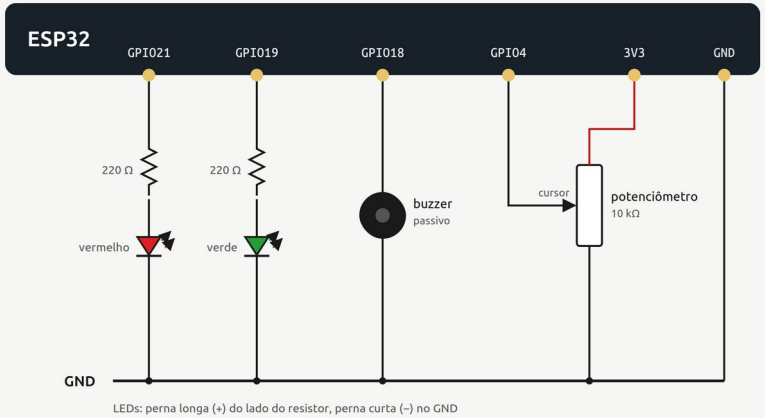
um kit por dupla

Hardware

- ESP32 **já com MicroPython**
- Cabo USB **de dados**
- Protoboard + fios (jumpers)
- 2 LEDs (vermelho e verde)
- 2 resistores de 220 Ω
- 1 potenciômetro (botão de girar)
- 1 buzzer passivo (faz som)

Software

- **Thonny** (editor gratuito)
- Pasta `exemplos/` com os códigos da aula



LEDs: perna longa (+) do lado do resistor, perna curta (–) no GND

Montagem do kit: 2 LEDs, 1 buzzer e 1 potenciômetro, todos no mesmo GND da placa

vamos montar isso juntos daqui a pouco

01

Conceitos

Microcontrolador, ESP32, MicroPython e seus módulos

19:00 · 15 min

O que é um microcontrolador?

Um **computador pequeno em um único chip**, feito para **controlar coisas**: ler sensores, decidir e acionar.

Seu notebook × Microcontrolador

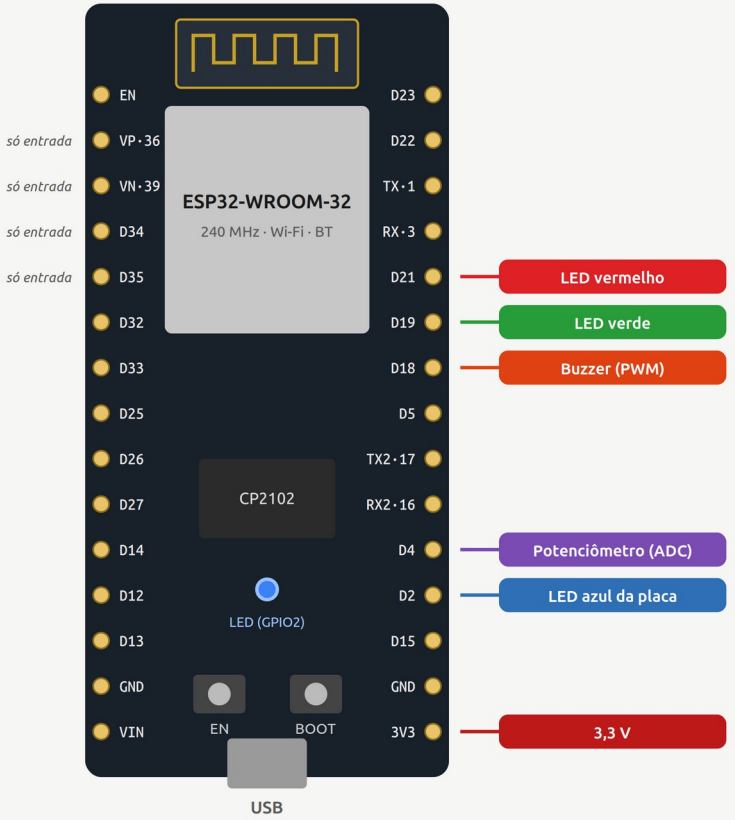
- GB de memória × **kB de memória**
- Vários programas × **um programa só**
- Tela e teclado × **pinos**

Onde estão

- Micro-ondas e controle remoto
- Carros e máquinas de lavar
- Casas inteligentes (**IoT**)

Conheça a ESP32

- Microcontrolador **barato** e muito usado
- Tem **Wi-Fi** e **Bluetooth**
- **GPIO** = pino que o programa liga, desliga ou lê
- Os pinos trabalham com **3,3 V**
- Hoje usamos só os pinos do **lado direito**



Pinagem da ESP32 DevKit V1 (30 pinos) — só pinos do lado direito

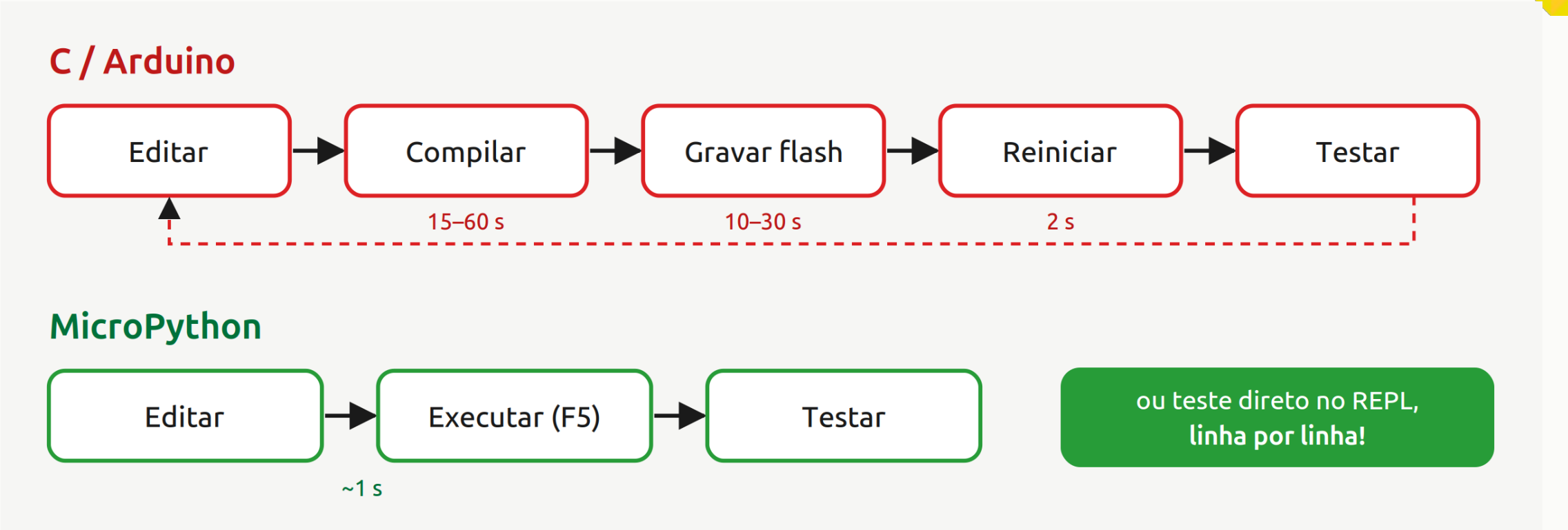
O que é o MicroPython?

É o **Python** rodando **dentro da placa**.

- Mesma sintaxe do Python
- Módulo `machine` para mexer nos pinos
- Já está instalado na placa: você só escreve o `.py`



Por que MicroPython?



Sem compilar: **escreveu, rodou, viu o resultado**.

Os módulos que vamos usar

cada um ganha um slide de referência ao longo da aula

`machine.Pin`

Liga, desliga e lê pinos digitais (0 ou 1)

→ **LEDs**

`machine.PWM`

Liga e desliga muito rápido: brilho e som

→ **LED verde e buzzer**

`machine.ADC`

Transforma uma tensão em número

→ **potenciômetro**

`time`

Esperar (sleep) e medir o tempo (ticks)

→ **em todo exemplo**

02

Thonny e o REPL

Instalando, configurando e conversando com a placa

19:15 · 25 min

Instalando o Thonny

editor gratuito, feito para quem está começando

Windows e macOS

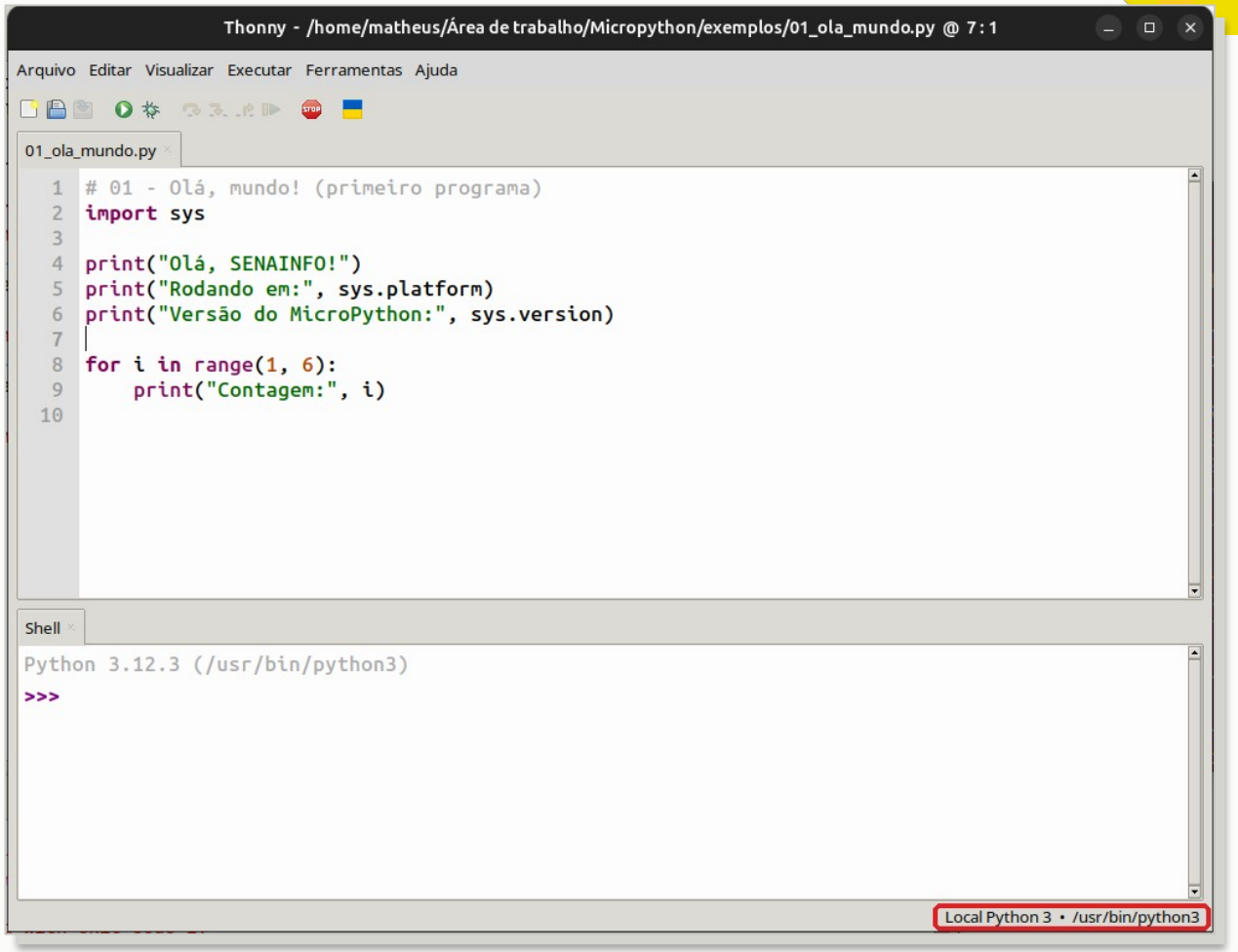
- Baixe o instalador em **thonny.org**
- Instale o driver USB da placa: **CP210x** ou **CH340**

Linux

- `sudo apt install thonny`
- ou `pip install thonny`

Na primeira vez

O Thonny roda o **Python do computador**, não o da placa.

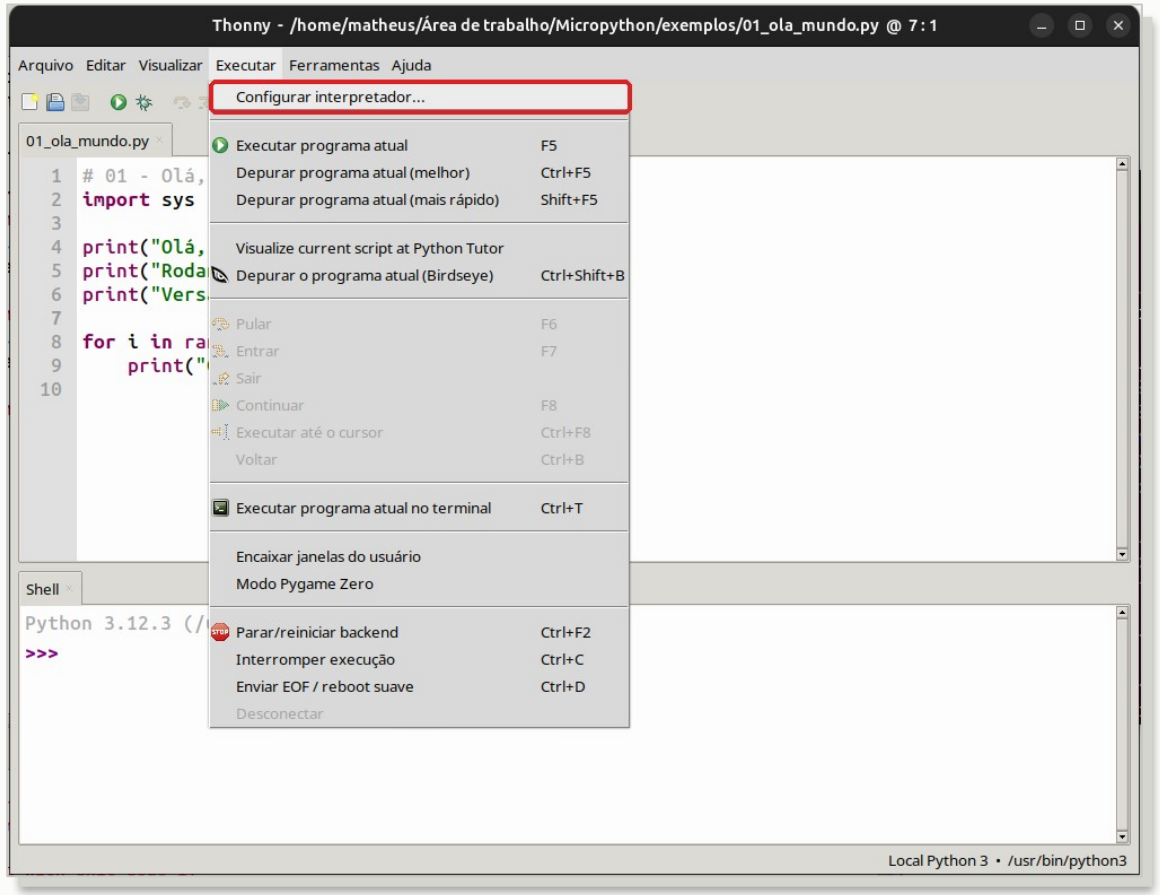


canto inferior direito: “Local Python 3” = ainda não está usando a ESP32

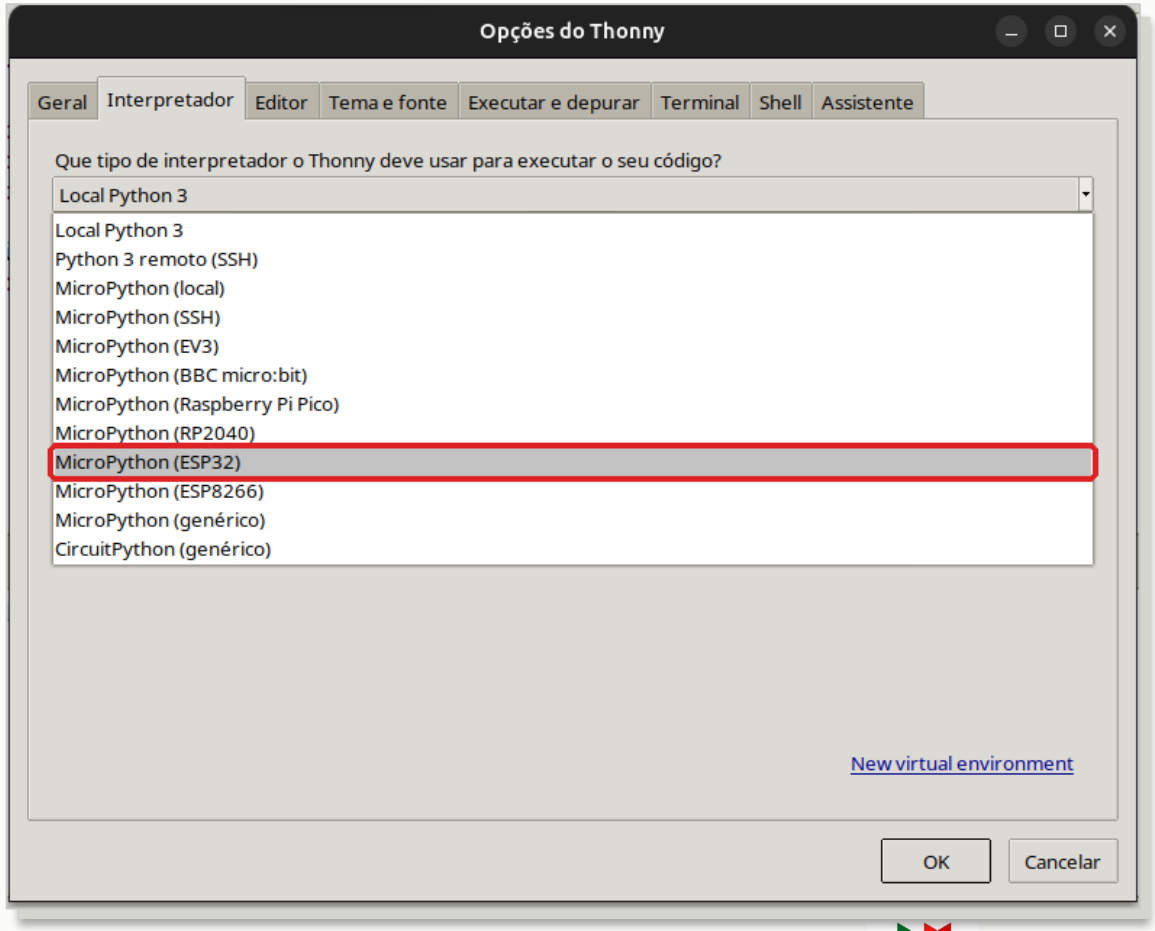
Configurando o interpretador (1/2)

faça uma única vez, com a placa conectada no USB

1 Menu **Executar → Configurar interpretador…**

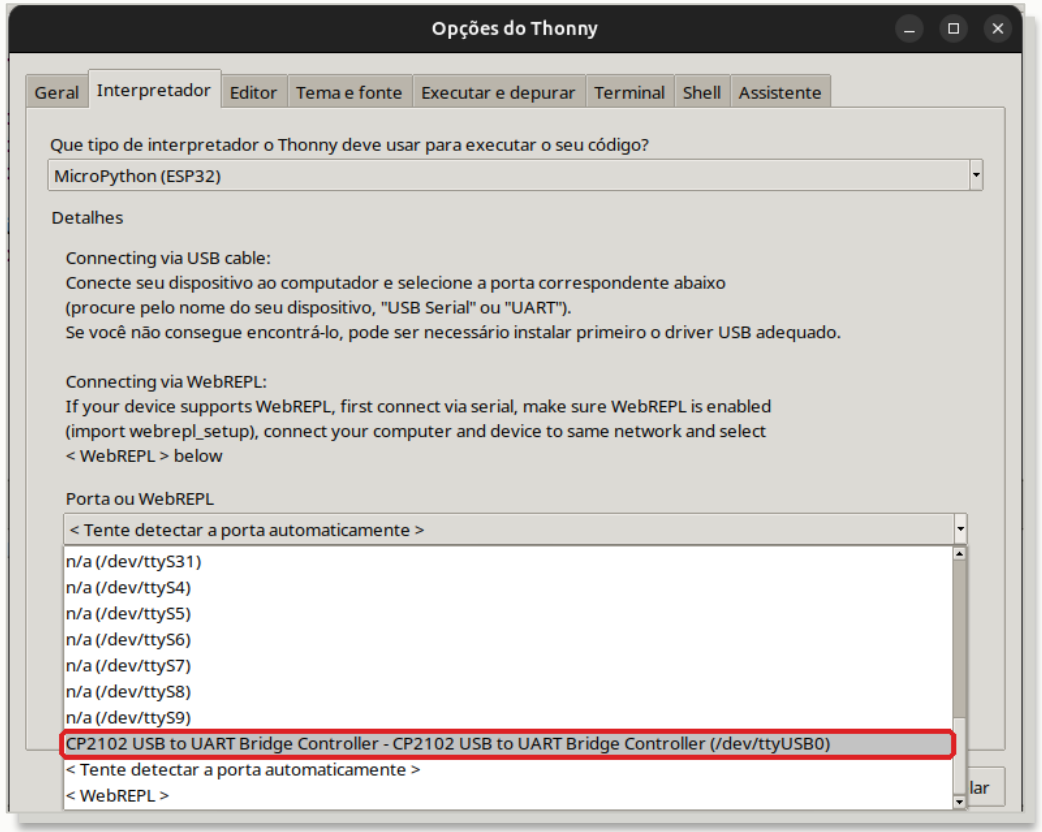


2 Na lista, escolha **MicroPython (ESP32)**

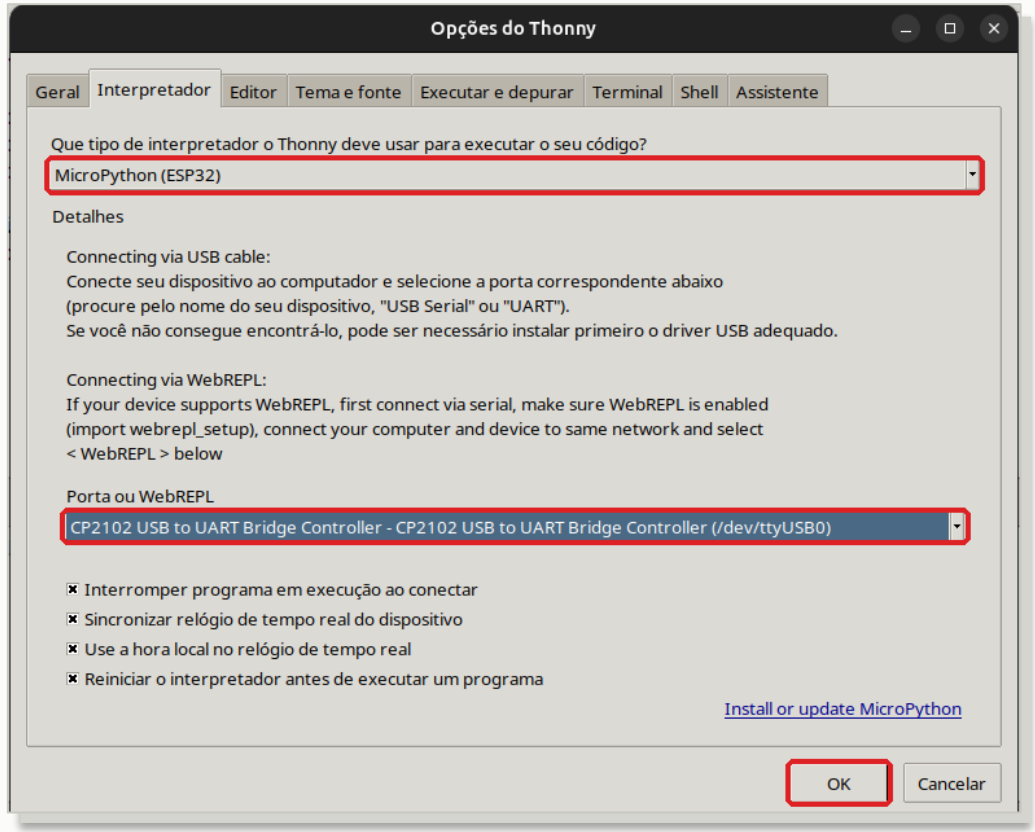


Configurando o interpretador (2/2)

3 Porta: **CP2102**, **CH340** ou **USB Serial**

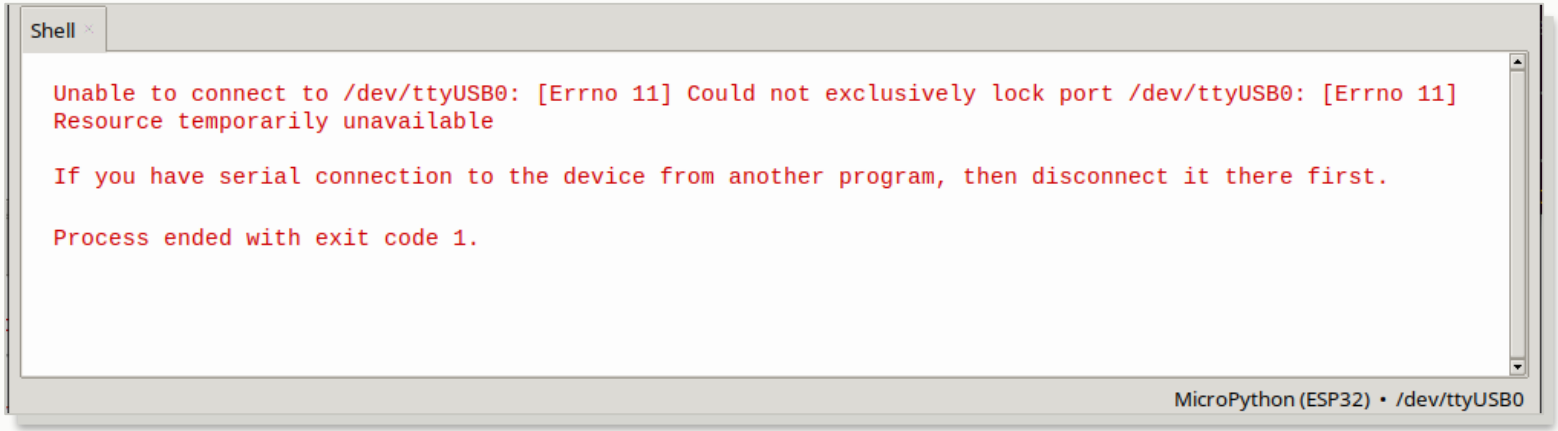


4 Confira e clique em **OK**



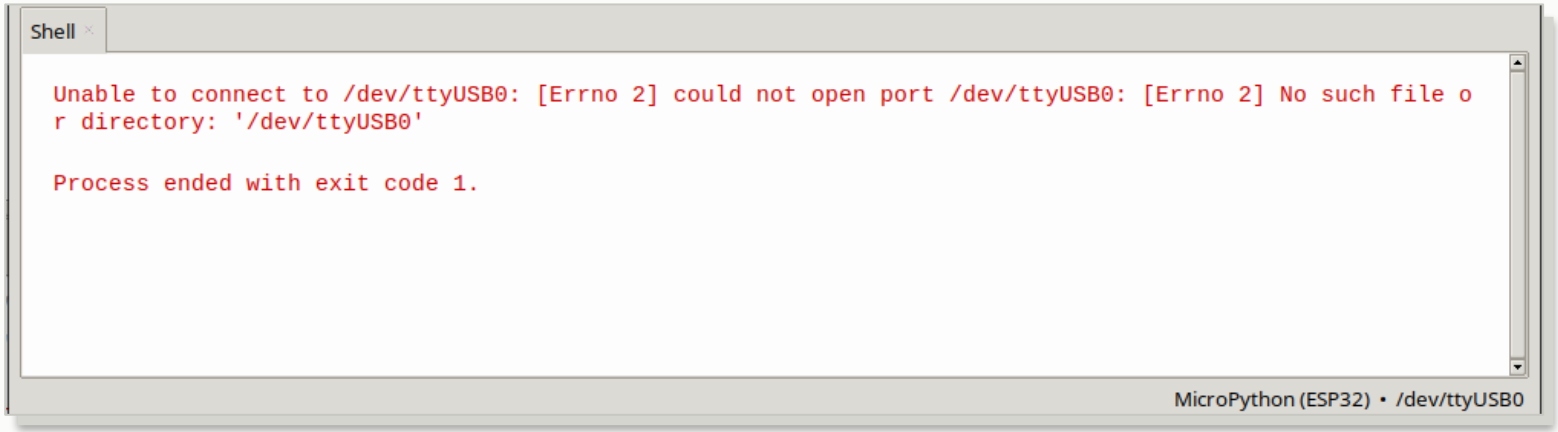
Deu certo: o Shell mostra **MicroPython v1.xx** e o prompt >>>, e o canto inferior direito mostra **MicroPython (ESP32)**.

Não conectou? Os dois erros mais comuns



Porta ocupada

Outro programa está usando a placa: feche o outro Thonny ou a Arduino IDE.

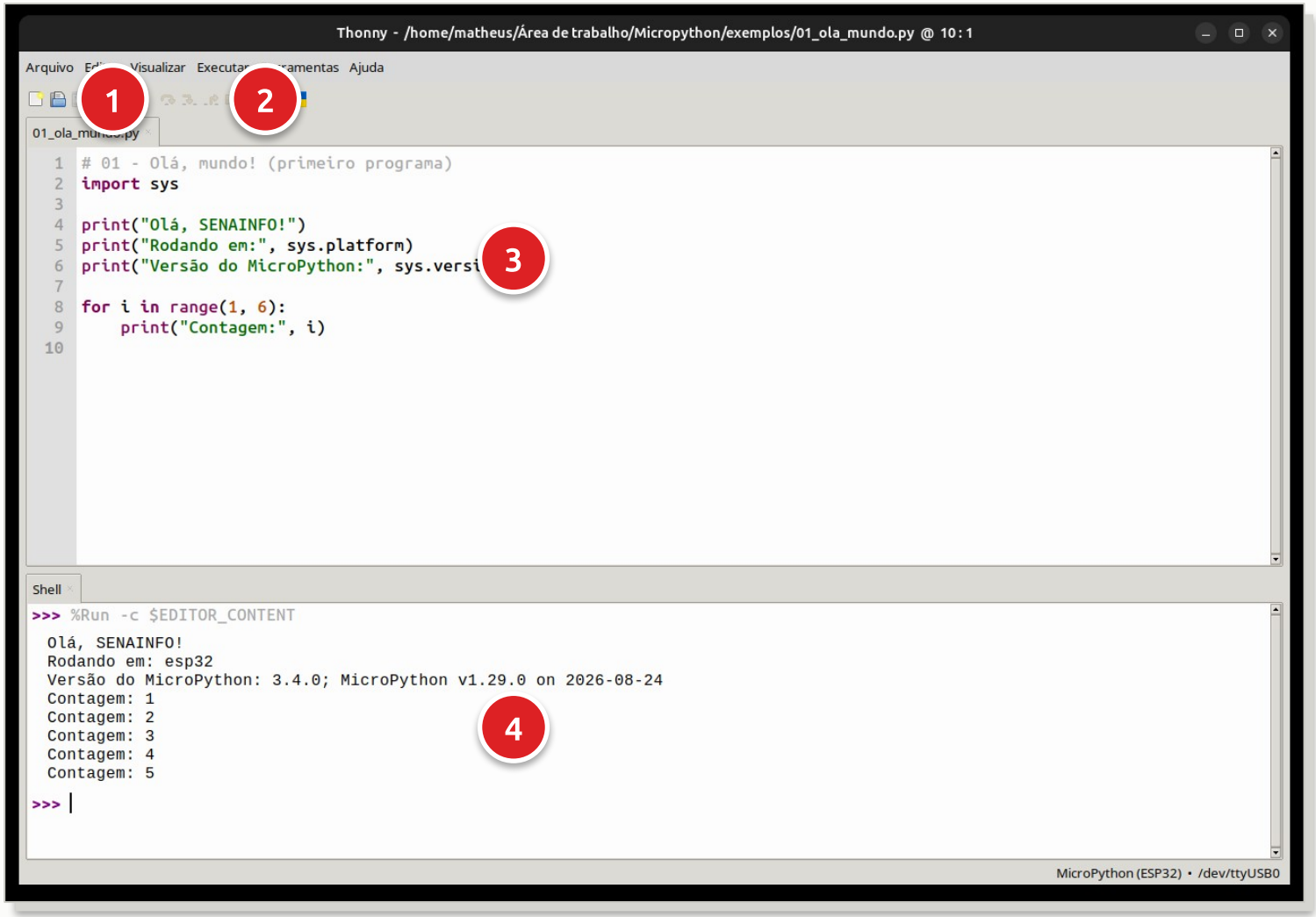


Placa não encontrada

O computador não vê a placa: troque o **cabo** ou instale o **driver**.

Resolveu? Clique em **STOP** (Ctrl+F2) para o Thonny tentar conectar de novo.

Tour pelo Thonny



1. **Executar (F5)**
   roda o programa na placa

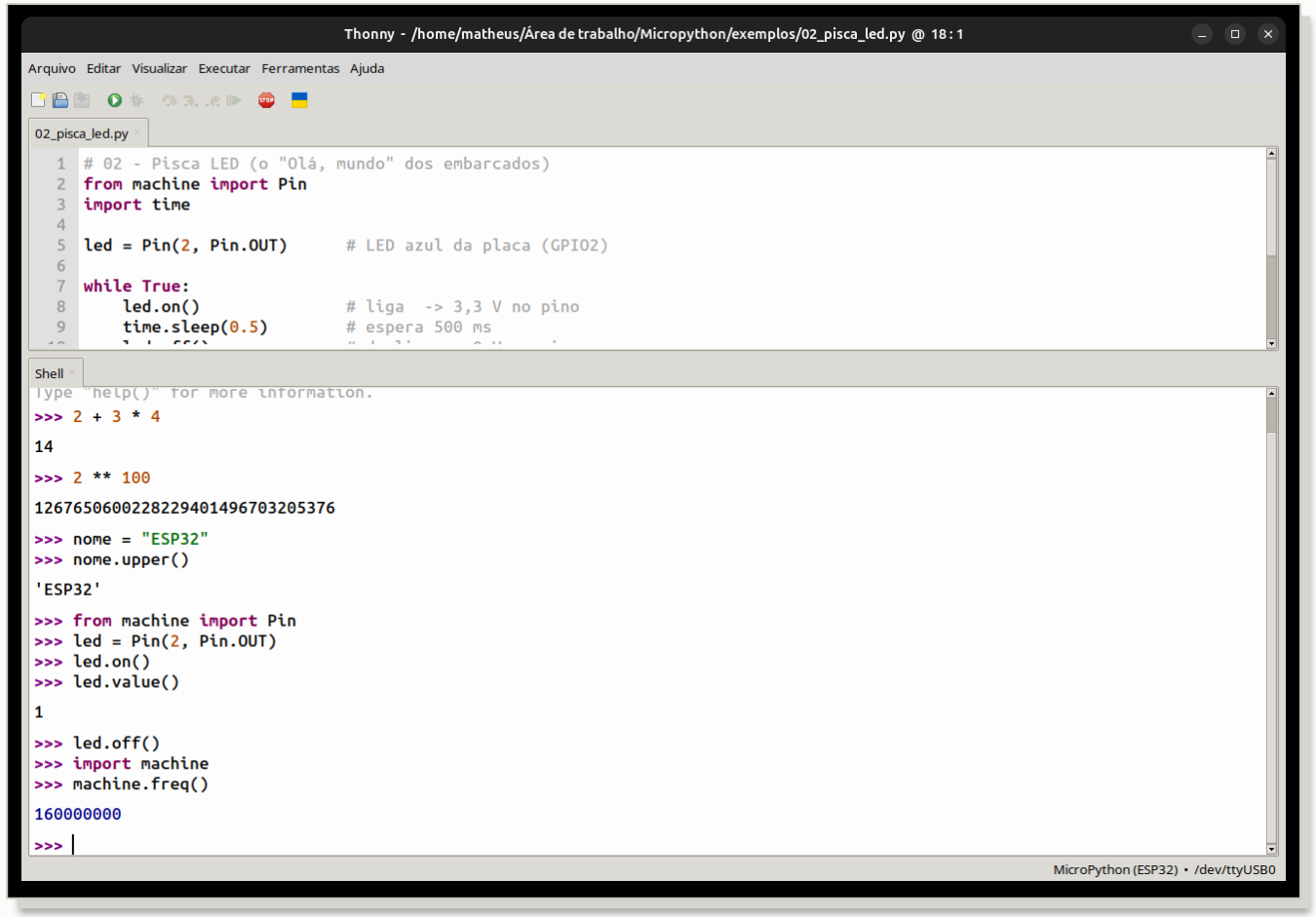
2. **Parar (STOP)**
   interrompe o programa

3. **Editor**
   onde você escreve o código

4. **Shell**
   mostra os print() e o prompt >>>

REPL: a placa responde na hora

cada linha roda assim que você aperta Enter



✋ Digite no Shell

```
from machine import Pin
led = Pin(2, Pin.OUT)
led.on()  👀 olhe a placa!
led.off()
```

- ↑ repete o comando anterior
- **Tab** completa nomes

Revisão rápida de Python

só o que vamos usar hoje

python_basico.py

```python
pino = 2                # variável
leds = [21, 19]         # lista

for p in leds:          # repete para cada item
    print("pino", p)

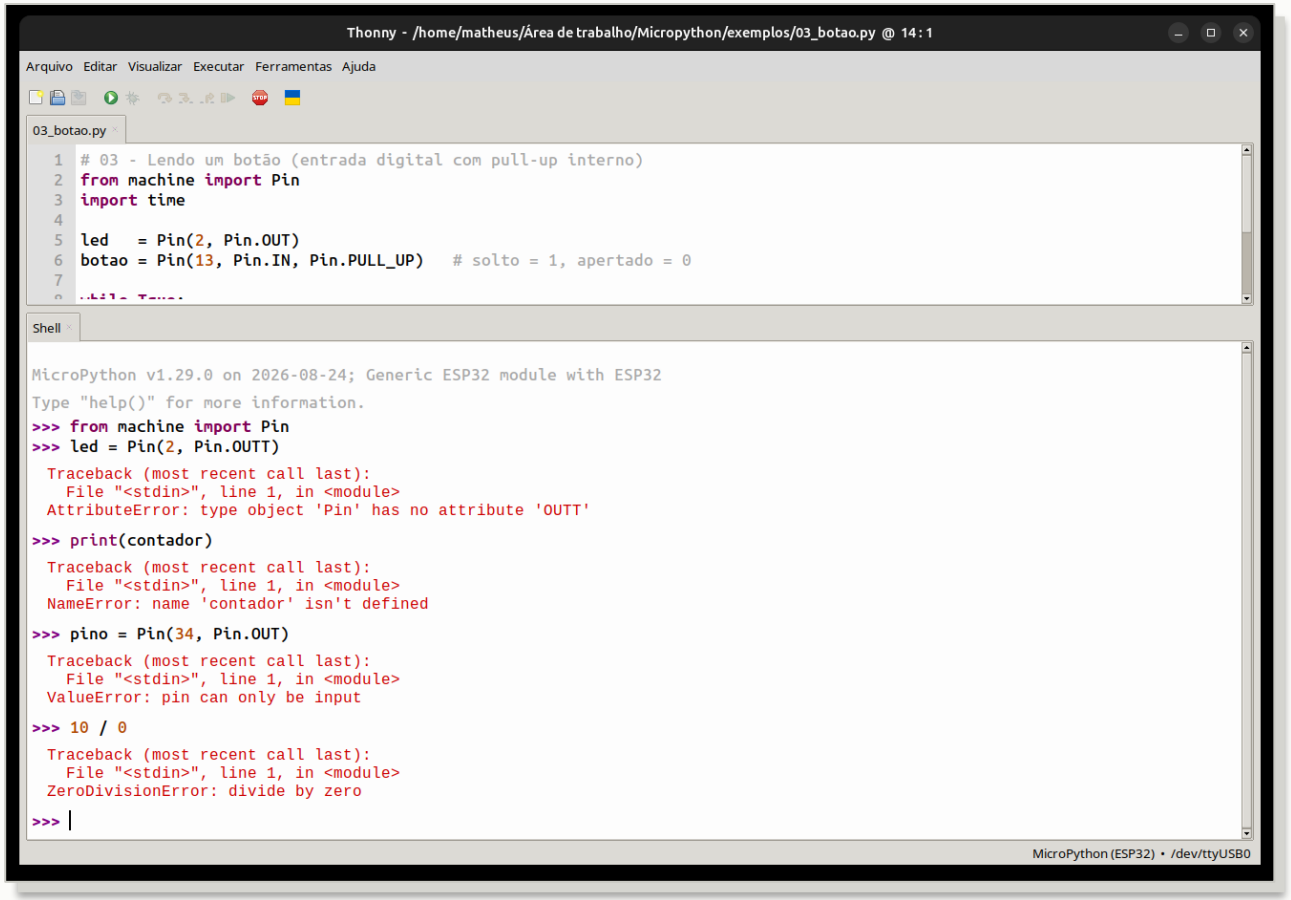
while True:             # repete para sempre
    print("oi")
```

```python
if pino == 2:
    print("LED da placa")
else:
    print("outro LED")

def dobro(x):
    return 2 * x
```

Atenção: em Python, a **indentação** (4 espaços) define o que está dentro do `if`, do `for` e da função.

Deu erro? Leia a última linha



`NameError`

nome escrito errado ou não criado

`ValueError`

valor que não serve (ex.: pino errado)

`IndentationError`

espaços errados no começo da linha

A **última linha** diz o tipo do erro; `line N` diz onde.

03

Pinos digitais

Montagem do circuito e o módulo machine.Pin

19:40 · 20 min

Eletrônica em 1 minuto

o mínimo para não queimar nada

Digital

- Pino **ligado** = 3,3 V = **1**
- Pino **desligado** = 0 V = **0**
- **GND** é o 0 V (o “negativo”)

LED

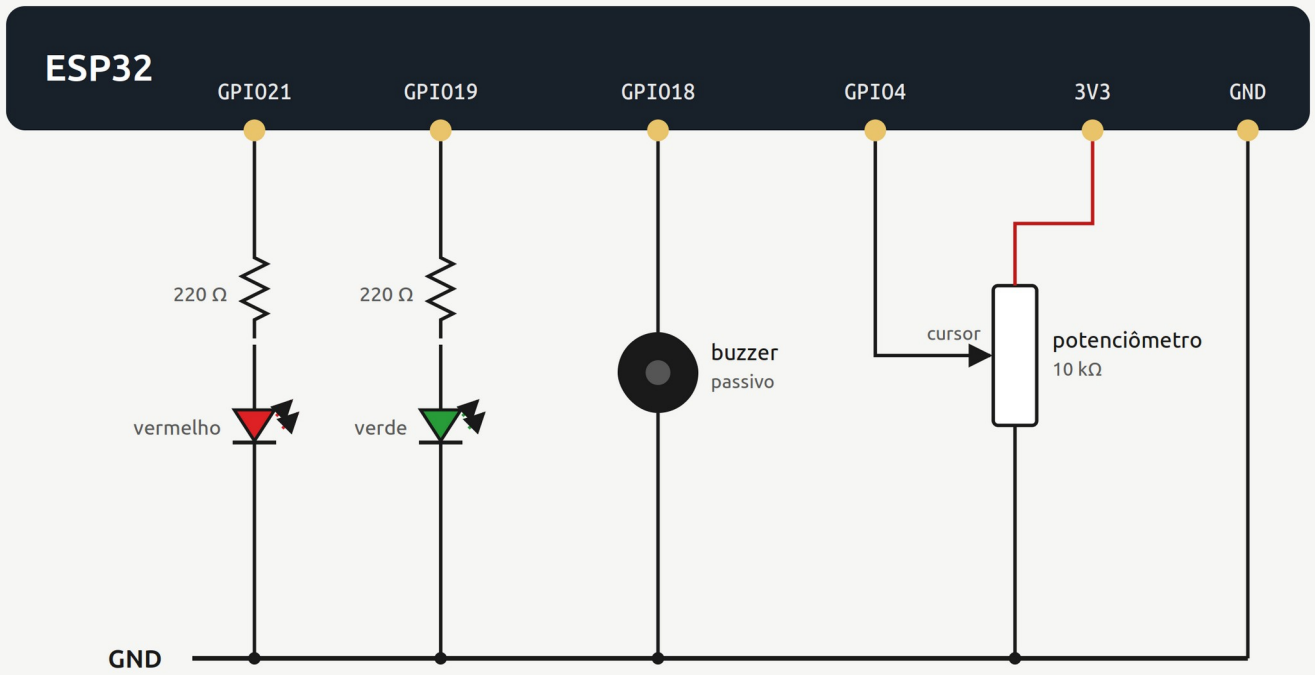
- Só acende num sentido
- **Perna longa** vai para o lado do pino
- **Perna curta** vai para o GND

Resistor

- **Limita a corrente** para o LED não queimar
- Usamos **220 Ω**
- Não tem lado certo

Montagem do circuito

monte tudo agora — vale para o resto da aula



LEDs: perna longa (+) do lado do resistor, perna curta (–) no GND

Montagem do kit: 2 LEDs, 1 buzzer e 1 potenciômetro, todos no mesmo GND da placa

- Monte com a placa **desconectada** do USB.

✋ Ligações

- LED vermelho → **D21**
- LED verde → **D19**
- (cada LED com seu resistor)
- Buzzer: **D18** e **GND**
- Potenciômetro: **3V3**, **D4** (pino do meio) e **GND**
- Tudo do **lado direito** da placa

Módulo machine.Pin

saídas e entradas digitais

referencia_pin.py

```python
from machine import Pin

led = Pin(21, Pin.OUT)       # pino de SAÍDA
led.on()                     # 3,3 V no pino
led.off()                    # 0 V no pino
led.value(1)                 # o mesmo que on()
led.value(not led.value())   # inverte o estado

ent = Pin(15, Pin.IN, Pin.PULL_UP)  # ENTRADA
ent.value()                  # lê 0 ou 1
```

Modos

- `Pin.OUT`: saída (LED, relé, buzzer ativo)
- `Pin.IN`: entrada (botão, sensor digital)
- `Pin.PULL_UP`: entrada lê 1 quando nada está ligado

Cuidados

- No máximo **3,3 V** em qualquer pino
- Corrente pequena por pino: **resistor** no LED

Pisca LED

exemplos/02_pisca_led.py

02_pisca_led.py

```python
# 02 - Pisca LED (o "Olá, mundo" dos embarcados)
from machine import Pin
import time

led = Pin(2, Pin.OUT)      # LED azul da placa (GPIO2)

while True:
    led.on()               # liga  -> 3,3 V no pino
    time.sleep(0.5)        # espera 500 ms
    led.off()              # desliga -> 0 V no pino
    time.sleep(0.5)
```

- `Pin(2, Pin.OUT)`: pino 2 é **saída**
- `on()` / `off()`: liga e desliga
- `sleep(0.5)`: espera meio segundo
- `while True`: repete **para sempre**

✋ Desafio

- Pisque o LED **vermelho** (GPIO21)
- Alterne **vermelho** e **verde** (GPIO19), como um pisca-alerta
- Bônus: pisque “SOS” (3 curtos, 3 longos, 3 curtos)

04

PWM

Brilho e som com o módulo machine.PWM

20:00 · 15 min

Módulo machine.PWM

pulsos rápidos para simular valores intermediários

referencia_pwm.py

```
from machine import Pin, PWM

pwm = PWM(Pin(19), freq=1000, duty_u16=0)

pwm.freq(1000)        # pulsos por segundo (Hz)
pwm.duty_u16(32768)   # 0 ... 65535 -> 50 % ligado
pwm.duty_u16(0)       # sempre desligado
pwm.duty_u16(65535)   # sempre ligado
pwm.deinit()          # libera o pino
```

Dois ajustes

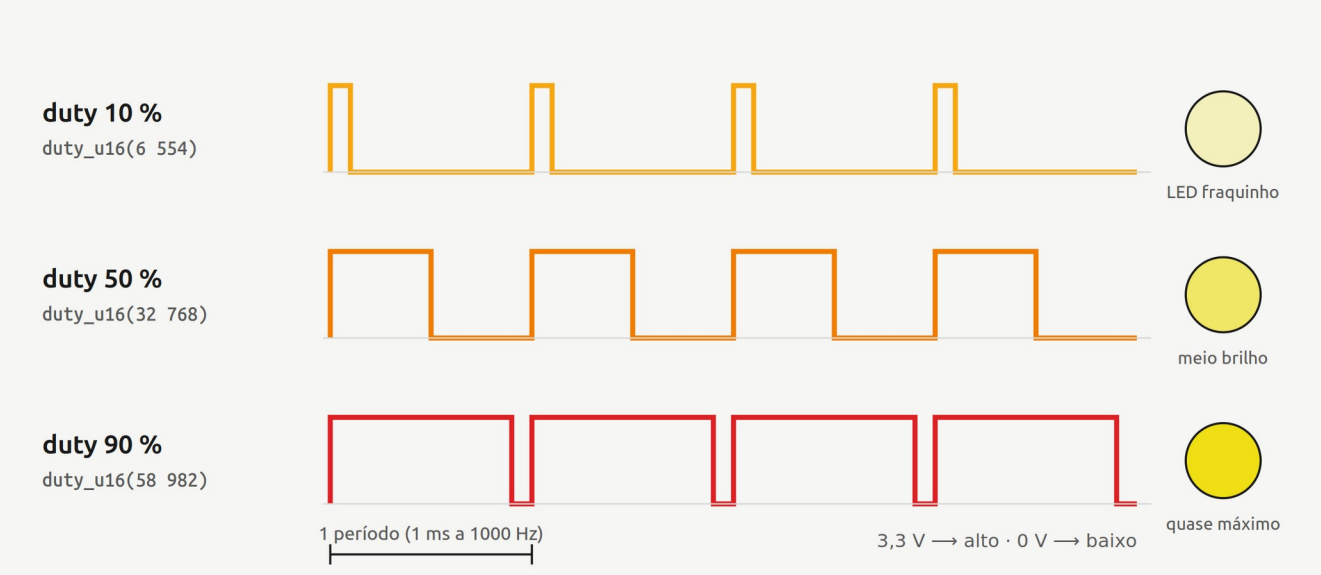
- `freq`: quantas vezes por segundo liga e desliga
- `duty`: que fração do tempo fica **ligado**

Uso típico

- **LED**: freq fixa (1000 Hz), muda o duty → **brilho**
- **Buzzer**: duty em 50 %, muda a freq → **nota**

PWM: controlando o brilho

liga e desliga tão rápido que o olho vê a média



- Mais tempo ligado = **mais brilho**
- `duty_u16(0)` → apagado
- `duty_u16(65535)` → brilho máximo

05_pwm_fade.py

```
led = PWM(Pin(2), freq=1000)          # 1000 Hz

while True:
    for brilho in range(0, 65536, 1024):      # sobe: 0 % -> 100 %
        led.duty_u16(brilho)
        time.sleep_ms(15)
```

✋ **Desafio**

› Faça o fade no LED **verde** (GPIO19)

PWM com som: música no buzzer

exemplos/06_buzzer.py

06_buzzer.py

```
from machine import Pin, PWM
import time

buzzer = PWM(Pin(18), freq=440, duty_u16=0)

NOTAS = {"C": 262, "D": 294, "E": 330, "F": 349,
         "G": 392, "A": 440, "B": 494, "-": 0}

musica = "C D E F - F F - C D C D - D D - C G F E - E E - C D E F - F F"

def tocar(nota, duracao_ms=200):
    if NOTAS[nota] == 0:
        buzzer.duty_u16(0)              # pausa = silêncio
    else:
        buzzer.freq(NOTAS[nota])
        buzzer.duty_u16(32768)          # 50 % de duty -> onda quadrada
    time.sleep_ms(duracao_ms)
    buzzer.duty_u16(0)
    time.sleep_ms(30)

for n in musica.split():
    tocar(n)
buzzer.deinit()
print("Fim da música!")
```

- A **frequência** define a nota (440 Hz = Lá)
- Um dicionário guarda as notas

✋ Desafio

› Mude a `musica` e toque outra melodia

Intervalo

10 minutos · voltamos às 20:25

Deixe o circuito montado: na volta vamos ler o potenciômetro.

05

ADC

Lendo o potenciômetro com o módulo machine.ADC

20:25 · 20 min

Módulo machine.ADC

conversor analógico → digital

referencia_adc.py

```
from machine import Pin, ADC

pot = ADC(Pin(4))          # GPIO4 (ADC2)
pot.atten(ADC.ATTN_11DB)   # faixa de 0 a ~3,3 V

pot.read_u16()             # 0 ... 65535
pot.read_uv()              # tensão em microvolts
```

Atenuação = faixa de leitura

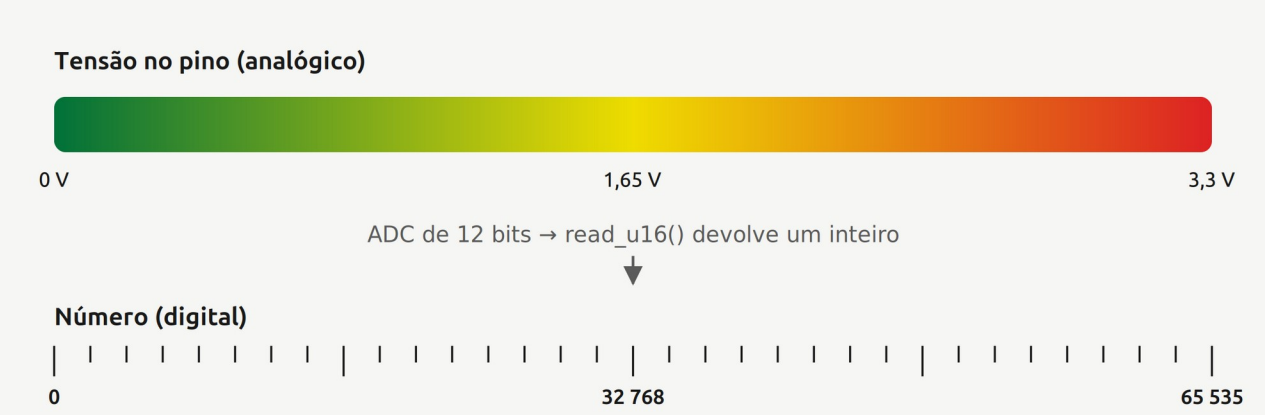
- `ATTN_0DB`: até ~1 V
- `ATTN_6DB`: até ~2 V
- `ATTN_11DB`: até ~3,3 V ← **usamos esta**

Atenção

- Nunca mais que **3,3 V** no pino
- A leitura **oscila** um pouco: é ruído, normal

Lendo o potenciômetro (ADC)

exemplos/07_potenciometro.py



07_potenciometro.py

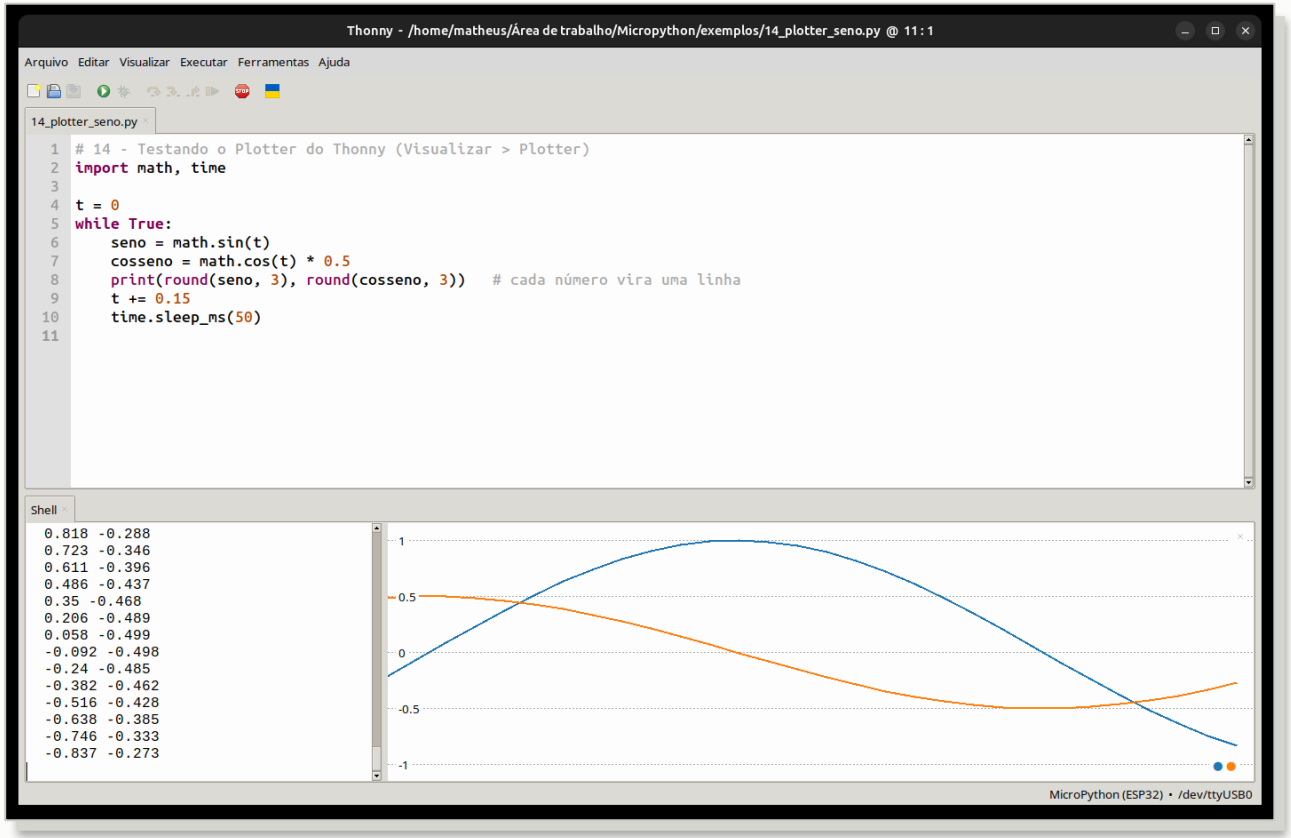
```
pot = ADC(Pin(4))                # GPIO4: entrada analógica (ADC2)
pot.atten(ADC.ATTN_11DB)         # lê de 0 a 3,3 V
led = PWM(Pin(2), freq=1000)

while True:
    valor = pot.read_u16()       # 0 ... 65535
    led.duty_u16(valor)          # brilho acompanha o potenciômetro
    print(valor)                 # o Plotter do Thonny desenha isso!
    time.sleep_ms(100)
```

- ADC transforma **tensão** em **número**
- 0 V → 0 · 3,3 V → 65 535
- Girou o potenciômetro → mudou o brilho

Vendo os dados em gráfico: o Plotter

Visualizar → Plotter



- Todo `print()` de números vira **gráfico**
- Ótimo para ver sensores **ao vivo**

✋ Faça agora

- Abra **Visualizar → Plotter**
- Rode o `07_potenciometro.py` e gire o pot

06

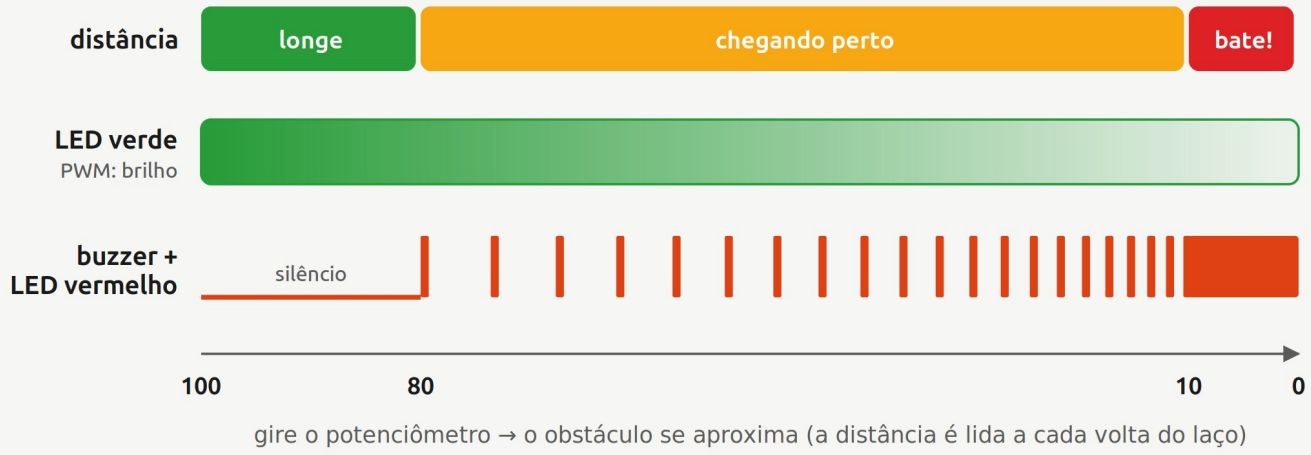
Prática final

Sensor de estacionamento: Pin + PWM + ADC

20:45 · 35 min

Prática final: sensor de estacionamento

exemplos/16_estacionamento.py



Como o sensor de ré de um carro: quanto mais perto, **mais rápido** o bipe.

✋ Faça agora

- Abra o `16_estacionamento.py`, rode e abra o **Plotter**
- Gire o potenciômetro devagar e observe LEDs e buzzer

Quem faz o quê

- `ADC`: o potenciômetro simula a **distância** do obstáculo
- `time`: a pausa entre bipes diminui quando chega perto
- `PWM`: brilho do LED **verde** e som do **buzzer**
- `Pin`: LED **vermelho** pisca junto com o bipe

Sensor de estacionamento: o código

exemplos/16_estacionamento.py

16_estacionamento.py (1/2)

```python
from machine import Pin, ADC, PWM
import time

pot      = ADC(Pin(4))          # "distância" do obstáculo
pot.atten(ADC.ATTN_11DB)        # lê de 0 a 3,3 V
verde    = PWM(Pin(19), freq=1000)
vermelho = Pin(21, Pin.OUT)
buzzer   = PWM(Pin(18), freq=2000, duty_u16=0)

def medir():                    # 0 = encostado, 100 = longe
    distancia = 100 - pot.read_u16() * 100 // 65535
    verde.duty_u16(distancia * 655)   # longe = verde forte
    print(distancia)                  # veja no Plotter
    return distancia

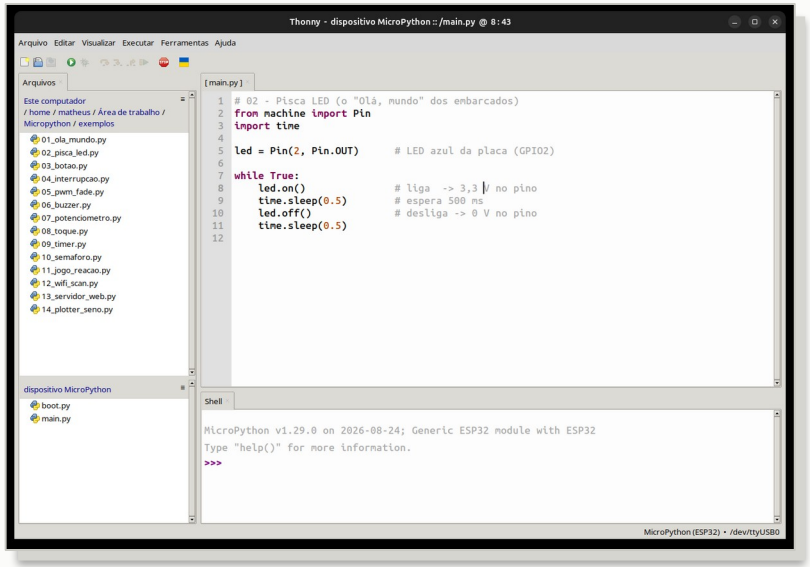
def bipe(som_ms, pausa_ms):
    vermelho.on()
    buzzer.duty_u16(32768)      # som
    time.sleep_ms(som_ms)
    vermelho.off()
    buzzer.duty_u16(0)          # silêncio
    time.sleep_ms(pausa_ms)
```

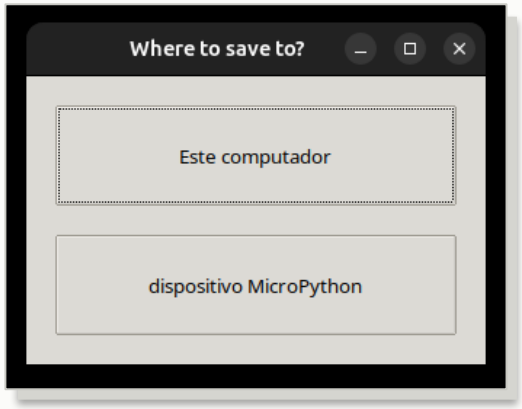
16_estacionamento.py (2/2)

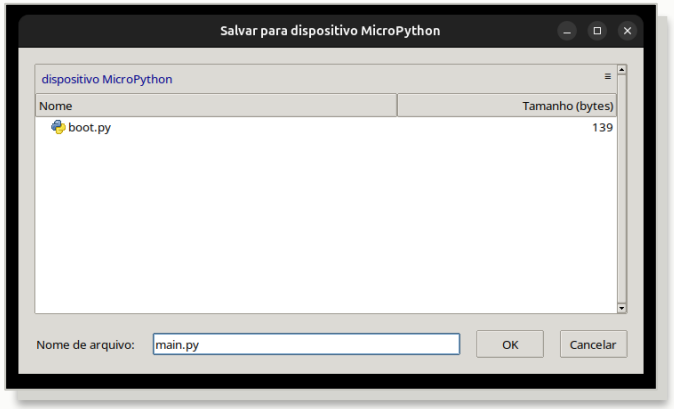
```python
while True:
    distancia = medir()
    if distancia > 80:          # longe: quieto
        time.sleep_ms(50)
    elif distancia > 10:        # perto: bipes
        bipe(60, distancia * 8)
    else:                       # vai bater!
        bipe(100, 0)
```

Fazendo a placa funcionar sem o computador









- **Arquivo → Salvar como → dispositivo MicroPython** → nome `main.py`
- Salve o sensor como `main.py`: ao ligar, a placa roda **sozinha**

Turbine o sensor!

escolha um desafio

- Junta tudo: **Pin**, **PWM** e **ADC**
- Mesma lógica de um sensor de ré de verdade
- Troque o potenciômetro por um sensor de distância e vira um projeto real

✋ Desafios

- Bipe **mais agudo** quando estiver perto (`buzzer.freq`)
- LED vermelho **aceso direto** na zona “bate!”
- Mude as zonas: alerta a partir de **50** em vez de 80
- Difícil: use as **notas** do `06_buzzer.py`, uma por zona

Cola rápida

guarde este slide

resumo.py

```python
from machine import Pin, PWM, ADC
import time

led = Pin(21, Pin.OUT)              # saída
led.on()
led.off()

pwm = PWM(Pin(19), freq=1000)       # brilho / som
pwm.duty_u16(32768)                 # metade

pot = ADC(Pin(4))                   # analógico
pot.atten(ADC.ATTN_11DB)
pot.read_u16()                      # 0 a 65535

time.sleep(1)                       # 1 segundo
time.sleep_ms(100)                  # 100 ms
```

Pinos do kit (lado direito)

- LED vermelho **21** · LED verde **19**
- Buzzer **18** · Potenciômetro **4**
- LED da placa **2**

Deu problema?

- Placa não aparece: troque o **cabo**
- “Device is busy”: aperte **STOP**
- Placa travada: aperte **EN**

Próximos passos

- Novos sensores: temperatura, distância, telas pequenas (OLED)
- Wi-Fi e páginas web com o módulo `network`
- Projeto: estação do tempo com página web
- Documentação: **docs.micropython.org**
- Pratique sem placa no simulador: **wokwi.com**

Obrigado!

Perguntas? Todo o material e os exemplos estão na apostila.

> **Antes de sair:**
> apague o `main.py` da placa e devolva o kit organizado 🙂